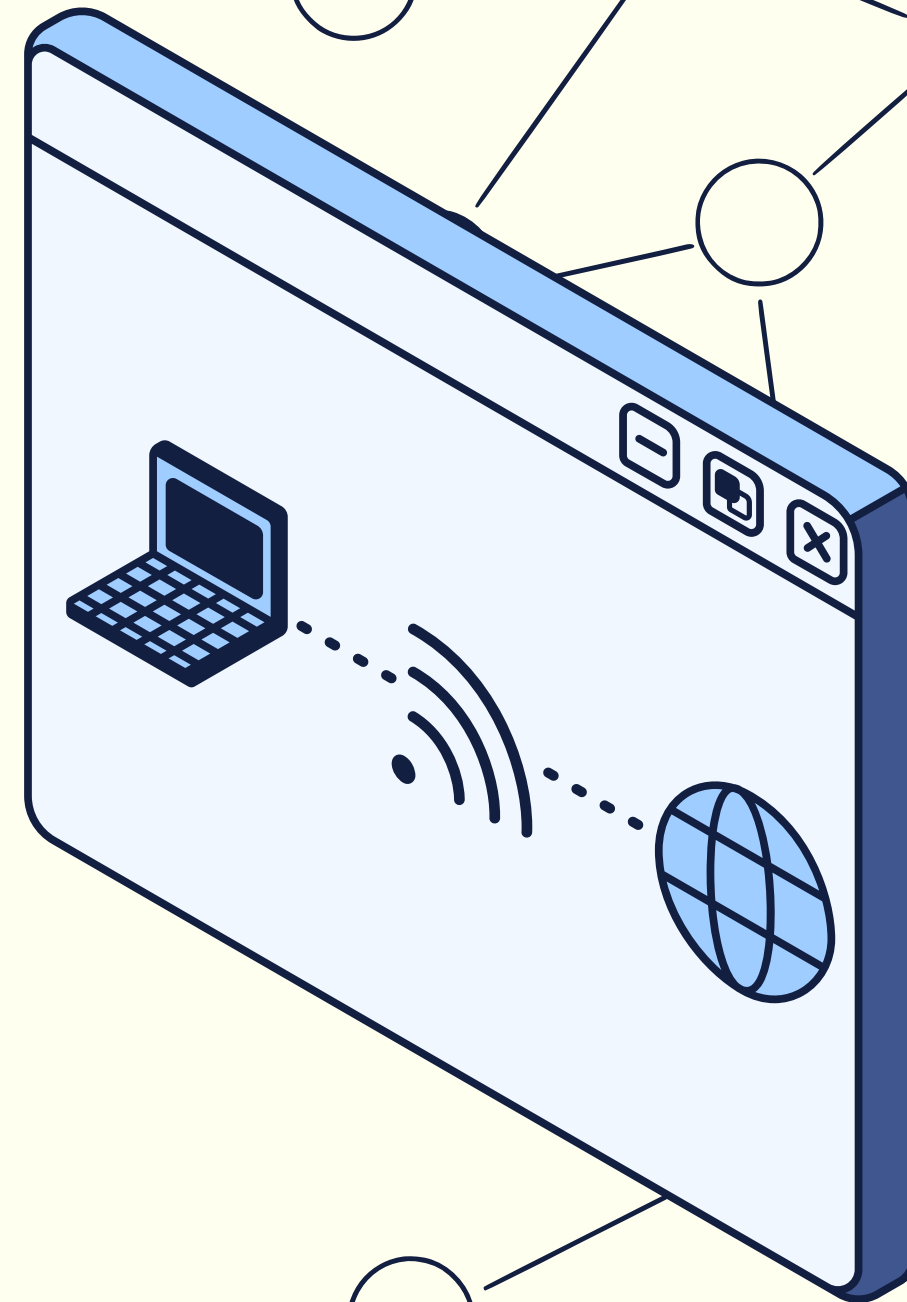


@DEVGUIO

APRENDENDO T.I COM O GUI!

BASE DE PROJETO REACT

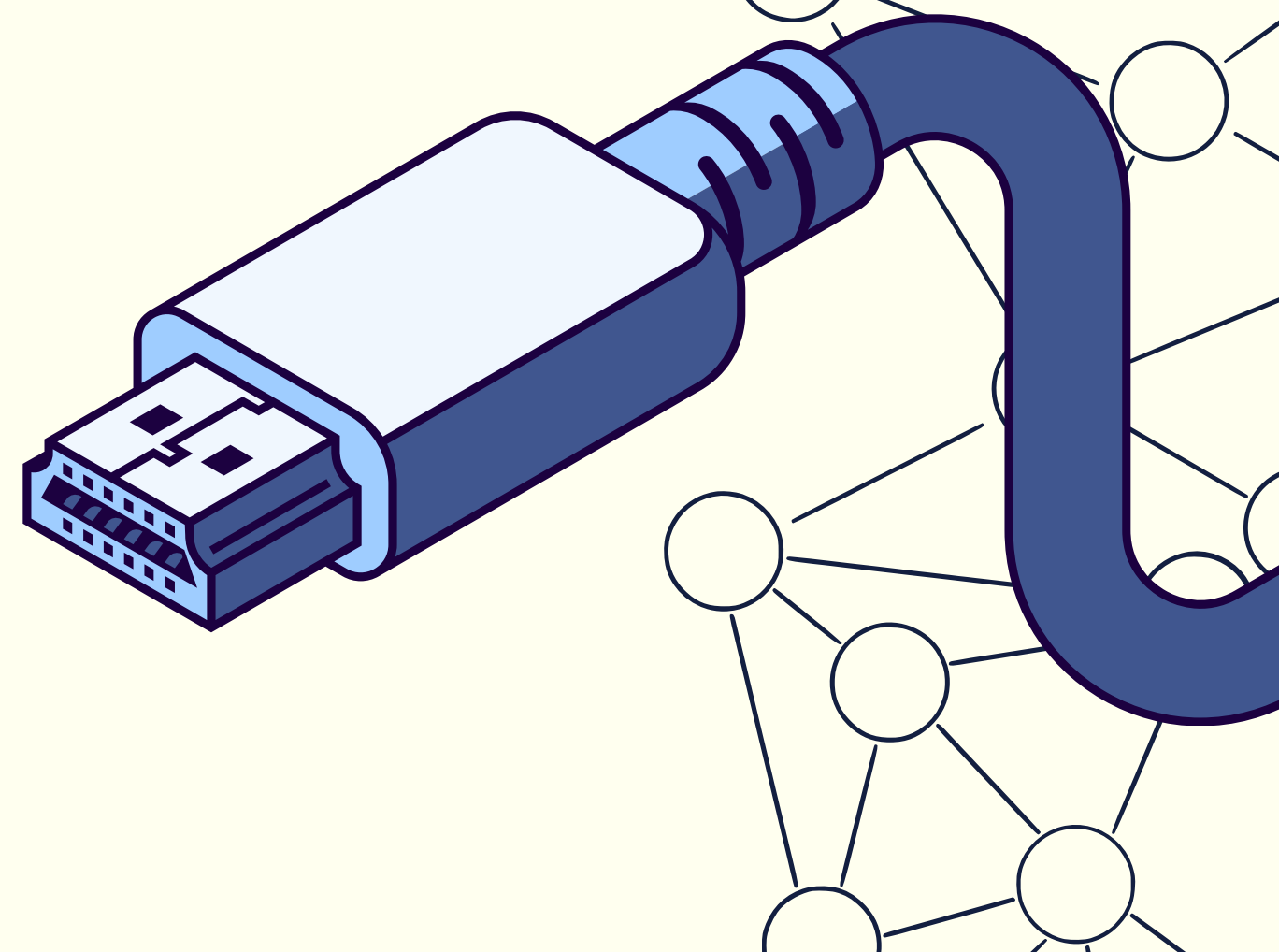


CONCEITOS FUNDAMENTAIS

REACT

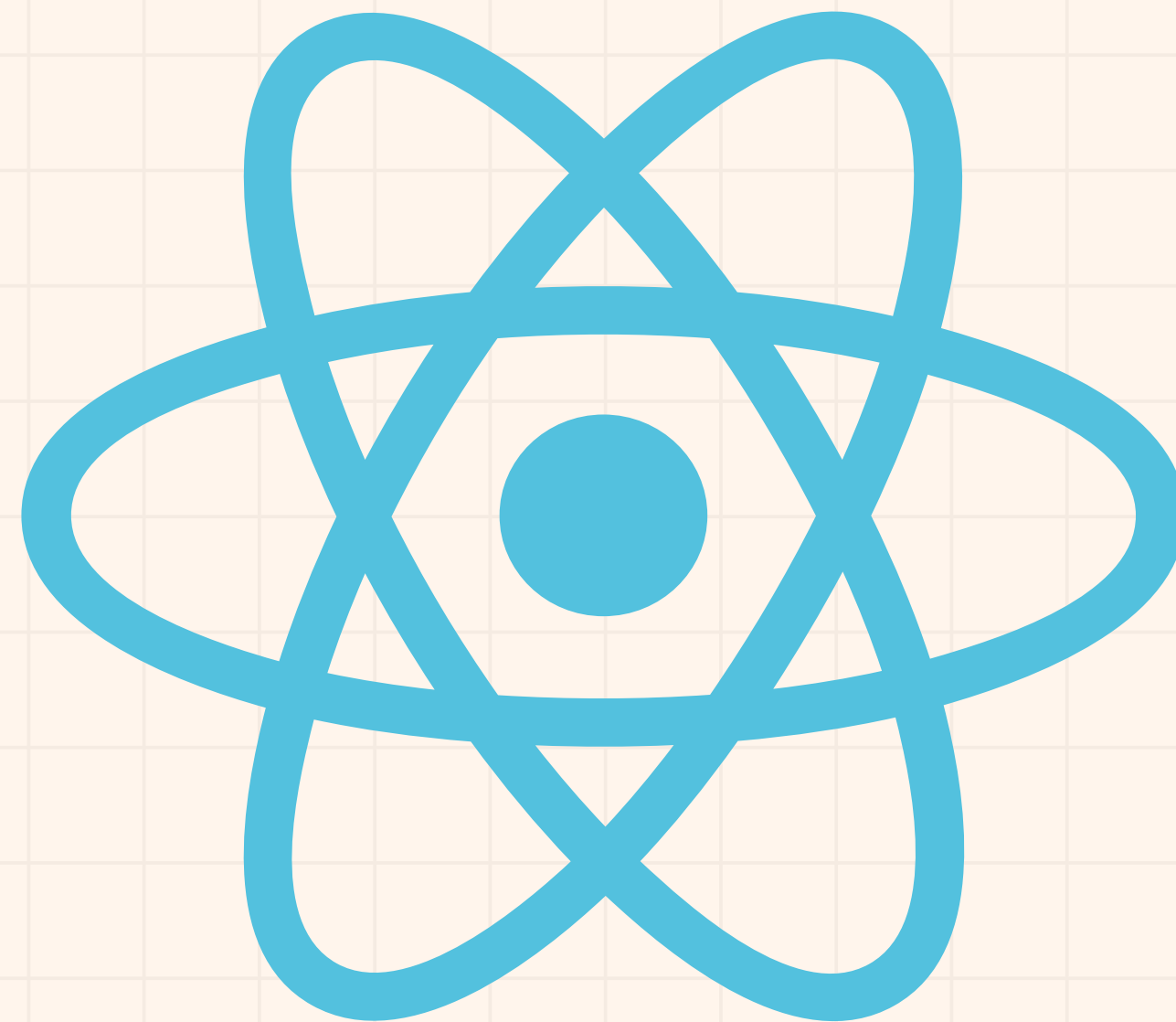
ENTENDENDO O PROJETO

POR QUE SERVE COMO BASE?



REACT

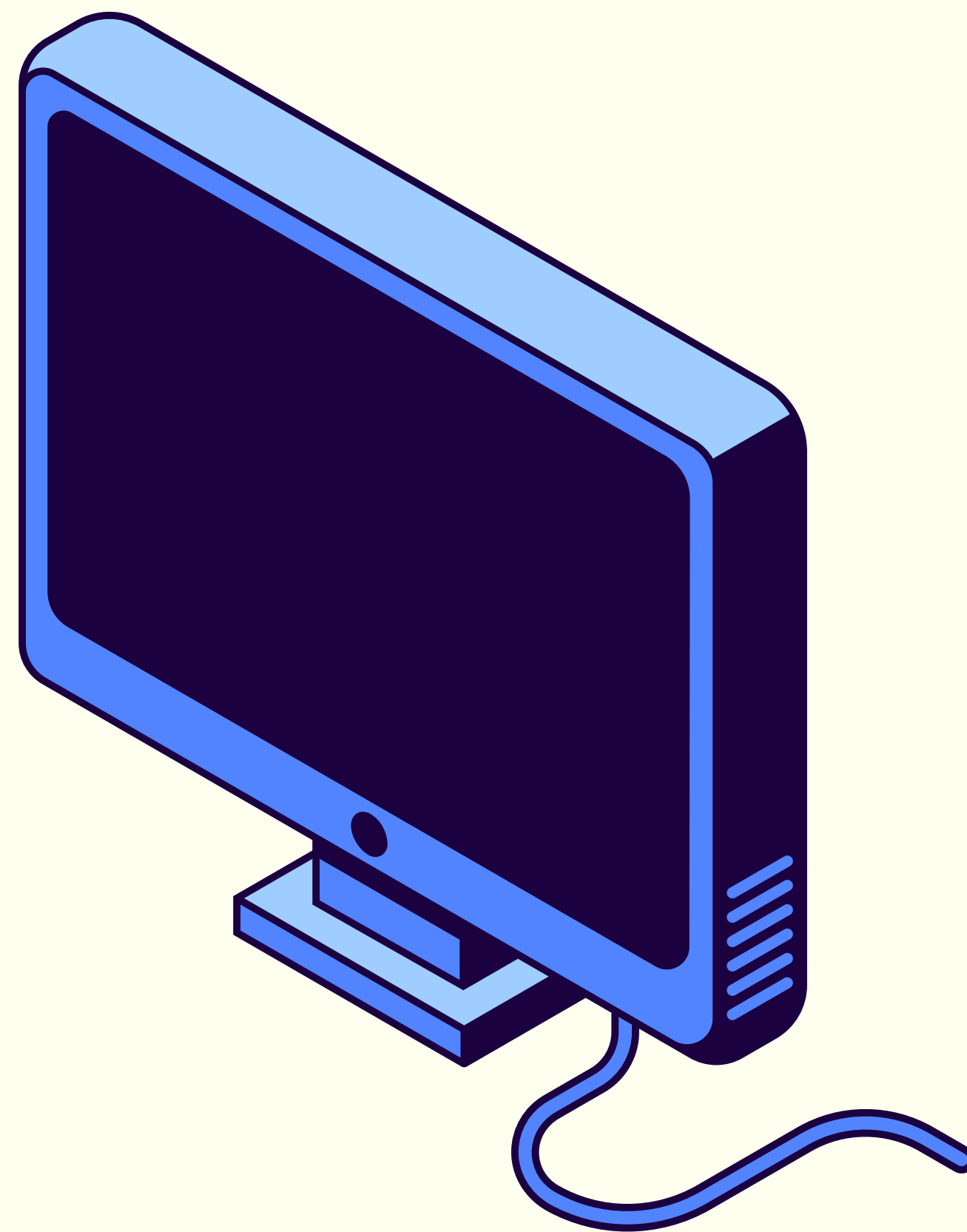
React é uma biblioteca JavaScript utilizada para a construção de interfaces a partir de peças denominadas <Componentes> Componentes são estruturas que podem carregar variáveis e e códigos HTML, viabilizando construções elaboradas de páginas web.



ENTENDENDO O PROJETO

Esse módulo é logo após a construção da API REST. Vamos pensar: eu já tenho uma API REST no back-end que sabe fazer coisas — cadastrar usuário, fazer login, buscar dados, etc. Mas ela só "fala" JSON, não tem cara. Essa base React é a interface visual que o usuário vai usar para interagir com tudo isso.

É como um restaurante: a cozinha (API) prepara os pratos, mas o cliente não vai até a cozinha pedir comida. O garçom (React) é a camada que conversa com o cliente, leva o pedido pra cozinha e traz o resultado de volta.

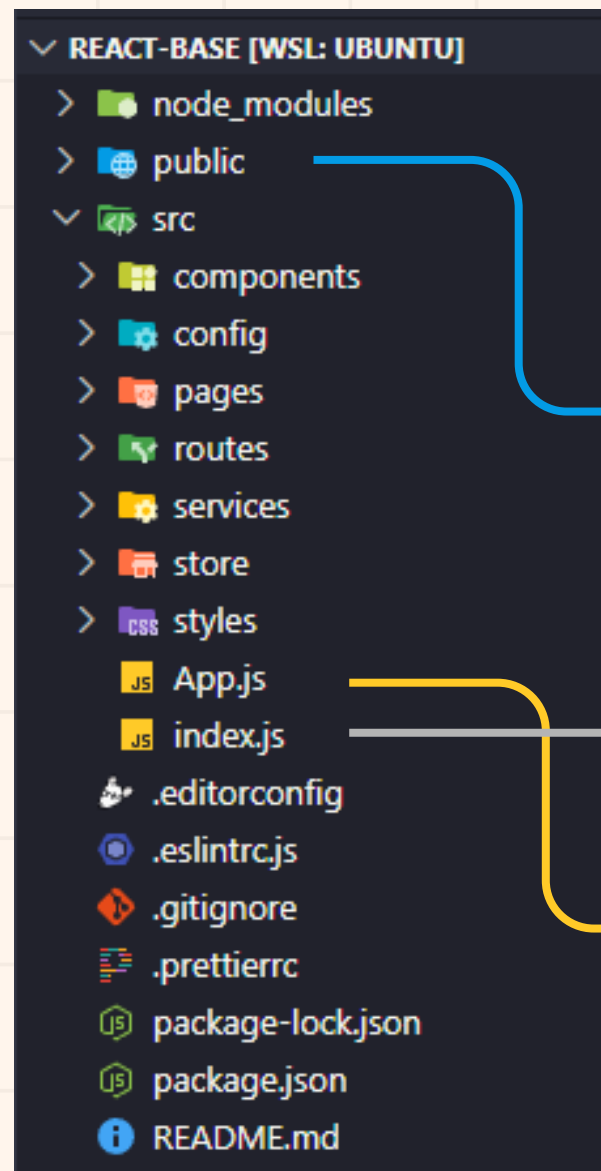


CAPÍTULO 1

O COMEÇO

Imagine que você está construindo um sistema de uma clínica médica. Os pacientes acessam pelo navegador para marcar consultas, fazer login, ver seu histórico. É uma aplicação real, com dados, usuários e uma API no back-end esperando as requisições.

O usuário digita a URL no navegador e aperta Enter. Nesse momento, o navegador baixa os arquivos do projeto e a primeira coisa que ele executa é o `index.js`. Esse arquivo é minúsculo, mas é a ignição do motor:



```
js
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Ele faz uma coisa só: pega aquele `<div id="root">` que existe no `public/index.html` — uma div vazia, um palco sem atores — e manda o React assumir o controle dela. A partir daqui, tudo que o usuário vê na tela é gerado pelo React, não pelo HTML diretamente.

O `<App />` é o ator principal chamado ao palco. E é no `App.js` que a história realmente começa a se montar.

1.1 O APP.JS

A ESTRUTURA

Se o index.js é a ignição, o App.js é a planta baixa do prédio inteiro. Olha o que ele faz:

```
js
function App() {
  return (
    <Provider store={store}>
      <PersistGate persistor={persistor}>
        <BrowserRouter history={history}>
          <Header />
          <Routes />
          <GlobalStyle />
          <ToastContainer autoClose={3000} />
        </BrowserRouter>
      </PersistGate>
    </Provider>
  );
}
```

Repara que é uma série de componentes aninhados um dentro do outro, como caixas russas. Cada camada envolve tudo que está dentro dela e adiciona um superpoder. Vamos de fora pra dentro:

● Camada 1 —

<Provider store={store}>

Essa é a camada mais externa. O **Provider** é como a rede elétrica do prédio — ele disponibiliza o Redux (o estado global da aplicação) para todos os cômodos. Qualquer componente lá dentro, não importa quão fundo esteja, pode acessar os dados do Redux. Sem esse Provider, o Redux simplesmente não funcionaria.

● Camada 2 —

<PersistGate persistor={persistor}>

Logo dentro, o **PersistGate** é o segurança da portaria. Ele segura a aplicação de renderizar até ter certeza que os dados salvos no **localStorage** (pelo Redux Persist) já foram carregados. Sabe quando você faz login, fecha o navegador e reabre — e continua logado? É esse cara que garante isso. Sem ele, a tela abriria antes dos dados chegarem e causaria comportamentos estranhos.

● Camada 3 —

<BrowserRouter history={history}>

Agora temos o **sistema de endereçamento do prédio**. O BrowserRouter é o que permite que a URL do navegador mude sem recarregar a página. Quando o usuário navega de **/login** para **/dashboard**, o React Router intercepta isso e troca os componentes na tela — sem fazer uma nova requisição ao servidor. O **history** é um objeto que permite navegar via código, por exemplo: após um login bem-sucedido, redirecionar o usuário automaticamente.

● Dentro do BrowserRouter —

os 4 filhos

Com toda a infraestrutura montada, finalmente temos o que aparece na tela:

- **<Header />** — o cabeçalho fixo que aparece em todas as páginas
- **<Routes />** — o sistema que decide qual página mostrar dependendo da URL
- **<GlobalStyle />** — os estilos CSS que valem para a aplicação inteira
- **<ToastContainer />** — o componente invisível que fica esperando para exibir notificações (aquelas mensagens de "Login realizado com sucesso!" que aparecem no canto da tela)

CAPÍTULO 2

O SISTEMA DE ENDEREÇOS

Nosso prédio da clínica médica já tem estrutura. Agora o paciente entrou pela porta, passou pela recepção — e precisa ser direcionado ao andar certo. Quem faz isso é o sistema de rotas.

O routes/index.js — A recepcionista

```
js
export default function Routes() {
  toast.success('Oie, sucesso');
  return (
    <Switch>
      <MyRoute exact path="/" component={Login} />
      <MyRoute path="*" component={Page404} />
    </Switch>
  );
}
```

O **Switch** funciona como uma recepcionista que olha pra URL e toma uma decisão: "Qual página devo mostrar agora?"

Ela lê as rotas de cima pra baixo e para na primeira que bater:

- Se a URL for exatamente / → mostra a página de Login
- Se for qualquer outra coisa (*) → mostra a página 404

O **exact** no primeiro item é importante — sem ele, a URL **/dashboard** também bateria com /, porque começa com /. O exact diz: "só vale se for exatamente esse endereço, sem nada a mais."

O toast.success('Oie, sucesso') é só um teste do professor pra mostrar que o ToastContainer está funcionando — aparece aquela notificação verde no canto da tela. Em produção você removeria isso.

O MyRoute.js — O segurança dos andares

Aqui está uma das peças mais inteligentes do projeto. Em vez de usar o **<Route>** padrão do React Router diretamente, o projeto criou um componente próprio chamado **MyRoute**.

Por quê?

Porque nem toda página pode ser acessada por qualquer um.

Na clínica, a sala de prontuários é restrita — só médicos autenticados entram.

O **MyRoute** é o segurança que fica na porta de cada sala:



```
js
export default function MyRoute({ component: Component, isClosed, ...rest }) {
  const isLoggedIn = false;

  if (isClosed && !isLoggedIn) {
    return (
      <Redirect to={{ pathname: '/login', state: { prevPath: rest.location.pathname } }} />
    );
  }

  return <Route {...rest} component={Component} />;
}
```

Ele recebe duas coisas importantes:

- **component**: qual página deve ser renderizada
- **isClosed**: essa rota é restrita? Precisa estar logado?

A lógica é simples: "Se a rota é fechada (**isClosed**) E o usuário não está logado (**!isLoggedIn**) → redireciona pro Login."

Repara no detalhe caprichado:

```
js
state: { prevPath: rest.location.pathname }
```

Ele guarda qual página o usuário tentou acessar antes de ser redirecionado. Assim, depois do login, dá pra mandar ele direto pra página que ele queria — em vez de sempre ir pro início. Igual a um shopping que anota sua mesa antes de te mandar pegar senha no caixa.

Como ficaria na prática

Quando o projeto crescer e tiver mais páginas, o **routes/index.js** vai ficar assim:

```
js
<Switch>
  <MyRoute exact path="/" component={Login} />
  <MyRoute exact path="/dashboard" component={Dashboard} isClosed />
  <MyRoute exact path="/pacientes" component={Pacientes} isClosed />
  <MyRoute path="*" component={Page404} />
</Switch>
```

As rotas com **isClosed** só abrem pra quem está logado. As sem **isClosed** são públicas. Simples e elegante.

Obs: o **isLoggedIn** está fixo como **false** por enquanto — é um placeholder. Quando o Redux estiver configurado, esse valor virá do estado global: "tem token salvo? então está logado."

O sistema de endereços está montado. No Capítulo 3 vamos ver as páginas em si — o que o usuário realmente **enxerga**.

CAPÍTULO 3

A SALA DE ESPERA

(A PÁGINA DE LOGIN)

Chegamos à primeira tela que o usuário vê. O paciente chegou à clínica. A recepcionista (o sistema de rotas) olhou pra URL / e disse: "você vai pra sala de espera — a página de Login." Agora vamos entender o que ele encontra lá.

O styled.js — A decoração da sala

Antes de entrar no **index.js**, precisa entender como essa página é estilizada, porque é uma abordagem diferente do CSS tradicional.

Antigamente você criaria um arquivo **Login.css** com classes e aplicaria no HTML. Com Styled Components, o estilo mora junto com o componente, escrito em JavaScript:

```
js

export const Title = styled.h1`
  background: green;
  small {
    font-size: 12pt;
    margin-left: 15px;
    color: #999;
  }
`;

export const Paragrafo = styled.p`
  font-size: 80px;
`;
```

Pensa assim: em vez de você criar uma tag **<h1>** e depois ir num arquivo CSS separado aplicar estilo nela, aqui você cria um componente chamado **Title** que já nasce estilizado. Ele é um **h1** por dentro, mas com identidade própria.

A vantagem? O estilo de **Title** nunca vai vazar pra outro componente em outra página. Cada componente cuida do próprio visual — sem conflito de classes CSS, sem efeitos colaterais. É como decorar cada sala da clínica de forma independente, sem risco de a tinta de uma sala escorrer pra outra.

O Login/index.js — O que acontece na sala

```
js
export default function Login() {
  const dispatch = useDispatch();

  function handleClick(e) {
    e.preventDefault();
    dispatch(exampleActions.clicaBotaoRequest());
  }

  return (
    <Container>
      <Title>Login <small>Oie</small></Title>
      <Paragrafo>Lorem ipsum dolor sit amet.</Paragrafo>
      <button type="button" onClick={handleClick}>Enviar</button>
    </Container>
  );
}
```

O visual é simples — é uma base, um template. Mas repara que já tem muita coisa acontecendo nos bastidores.

Os componentes visuais

O **Container** vem do **GlobalStyles** — é um estilo de container que vale pra todas as páginas, não só o Login. O **Title** e o **Paragrafo** vêm do **styled.js** local, específicos dessa página.

O useDispatch — O interfone da sala

Aqui começa a tocar o Redux pela primeira vez. Olha essa linha:

```
js
const dispatch = useDispatch();
```

Imagina que o Redux é uma central de controle no subsolo do prédio. Cada sala não fala diretamente com ela — usa um interfone. O **dispatch** é esse interfone. Quando o usuário clica no botão, a sala de espera não resolve nada sozinha — ela envia uma mensagem pra central:

```
js
dispatch(exampleActions.clicaBotaoRequest());
```

Essa mensagem se chama action — é basicamente um aviso do tipo *"ei central, o botão foi clicado, faz o que precisa ser feito."* O que acontece depois disso é responsabilidade do Redux, não da página.

Por que isso é bom?

Porque a página de Login tem uma responsabilidade só: mostrar a interface e capturar o clique. Ela não sabe como fazer login, não chama a API, não salva token. Ela só avisa a central. Isso é separação de responsabilidades — cada parte do sistema faz uma coisa só, e faz bem feita.

É como o paciente na sala de espera: ele não vai até o médico sozinho, não acessa o prontuário, não agenda nada. Ele aperta o botão da recepção e avisa "estou pronto". O que acontece depois é com a clínica.

CAPÍTULO 4

A CENTRAL DE CONTROLE (REDUX)

O paciente apertou o botão na sala de espera. A mensagem foi enviada pelo interfone (**dispatch**). Agora vamos descer até o subsolo — a central de controle — e entender como ela funciona.

O store/index.js — Construindo a central

```
js
const sagaMiddleware = createSagaMiddleware();

const store = createStore(
  persistedReducers(rootReducer),
  applyMiddleware(sagaMiddleware)
);

sagaMiddleware.run(rootSaga);

export const persistor = persistStore(store);
export default store;
```

Esse arquivo monta a central do zero. Vamos por partes:

O store — O cofre de dados

O **store** é o estado global da aplicação — **um único objeto JavaScript que guarda tudo que a aplicação precisa saber**. Quantos usuários estão logados? O token JWT? Se tem erro na tela?

Pensa nele como a **ficha cadastral da clínica**: um único lugar onde ficam todos os dados dos pacientes, consultas, médicos. Qualquer sala do prédio pode consultar essa ficha — mas ninguém pode alterá-la diretamente. Só a central pode fazer isso, seguindo um processo.

O rootReducer – As regras de alteração

O reducer é quem define **como o estado muda** quando uma action chega. Recebe o estado atual e a action, e devolve o estado novo. Nunca modifica o original — sempre cria um novo. É como um cartório: você não rasura o documento original, você emite um novo com as alterações.

O sagaMiddleware — O segurança do corredor

O **applyMiddleware(sagaMiddleware)** instala um intermediário entre o **dispatch** e o **reducer**. Quando uma action é disparada, ela não vai direto pro reducer — passa primeiro pelo corredor das Sagas. Pensa assim: a mensagem do interfone chega na central, mas antes de ir pro cofre de dados, passa por um segurança que verifica: *"essa action precisa chamar a API antes? Então eu cuido disso primeiro."*

O persistedReducers e persistStore — A memória da clínica

Sem isso, toda vez que o usuário recarregasse a página, o Redux esqueceria tudo — como uma clínica que joga fora a ficha do paciente ao fim do dia. O Redux Persist resolve isso salvando o estado no **localStorage** automaticamente. O **persistStore** cria o **persistor** que o **PersistGate** lá no **App.js** usa para esperar esses dados carregarem antes de mostrar a tela.

O actions.js — Os tipos de mensagem do interfone

```
js
export function clicaBotaoRequest() {
  return { type: types.BOTAO_CLICADO_REQUEST };
}
export function clicaBotaoSuccess() {
  return { type: types.BOTAO_CLICADO_SUCCESS };
}
export function clicaBotaoFailure() {
  return { type: types.BOTAO_CLICADO_FAILURE };
}
```

Cada action é uma função que devolve um objeto simples com um **type** — o nome da mensagem. E repara que existem três para uma mesma operação: **REQUEST**, **SUCCESS** e **FAILURE**.

Esse padrão de três é muito importante. Pensa numa consulta médica:

- **REQUEST** → o paciente apertou o botão: *"quero marcar uma consulta"*
- **SUCCESS** → a consulta foi marcada com sucesso
- **FAILURE** → algo deu errado, médico indisponível

Enquanto o **REQUEST** está sendo processado, a aplicação pode mostrar um loading. Se vier **SUCCESS**, mostra uma mensagem de sucesso. Se vier **FAILURE**, mostra o erro. Esse ciclo cobre todos os estados possíveis de uma operação assíncrona.

O **types** é importado de um arquivo separado que provavelmente contém:

```
js
export const BOTAO_CLICADO_REQUEST = 'BOTAO_CLICADO_REQUEST';
export const BOTAO_CLICADO_SUCCESS = 'BOTAO_CLICADO_SUCCESS';
export const BOTAO_CLICADO_FAILURE = 'BOTAO_CLICADO_FAILURE';
```

Centralizar os nomes num arquivo **types.js** evita erros de digitação — se você errar o nome da action em dois lugares, o sistema quebra silenciosamente. Com o **types.js**, o erro aparece na importação.

A central está montada, os tipos de mensagem estão definidos. Agora falta entender **quem escuta essas mensagens e o que faz com elas** — esse é o trabalho das **Sagas** e dos **Reducers**.

CAPÍTULO 5

SAGAS E REDUCER

A mensagem saiu do interfone. Passou pelo corredor do sagaMiddleware. Agora vamos ver o que acontece nesse corredor — e o que acontece depois, no cartório.

O sagas.js — O Corredor Inteligente

```
js
function* exampleRequest() {
  try {
    yield call(requisicao);
    yield put(actions.clicaBotaoSuccess());
  } catch {
    toast.error('Deu erro. ');
    yield put(actions.clicaBotaoFailure());
  }
}

export default all([takeLatest(types.BOTAO_CLICADO_REQUEST, exampleRequest)]);
```

O function* — Uma função que pausa

Antes de tudo: esse asterisco não é enfeite. O **function*** cria uma Generator Function — um tipo especial de função que pode pausar no meio da execução e esperar algo acontecer antes de continuar.

Funções normais são como um carro sem freio — entram, executam tudo, saem. Uma Generator Function é como um carro com freio: ela avança, para, espera, avança de novo.

O **yield** é o freio. Cada vez que aparece, a função pausa e espera o resultado antes de continuar.

O fluxo dentro da Saga

```
js
yield call(requisicao);
```

O **call** chama uma função assíncrona e pausa a Saga até ela terminar. Aqui está sendo chamada uma função **requisicao** que simula uma chamada de API — ela espera 600ms e resolve. No projeto real com sua API REST, aqui ficaria o **axios.post('/login', dados)**.

```
js
yield put(actions.clicaBotaoSuccess());
```

O **put** é o equivalente ao **dispatch** dentro da Saga. Se a requisição funcionou, ele dispara a action de **SUCCESS** — avisando o resto do sistema que deu tudo certo.

Se algo der errado, o **catch** dispara a action de **FAILURE** e mostra um toast de erro na tela.

O takeLatest — O filtro anti-spam

```
js
export default all([takeLatest(types.BOTAO_CLICADO_REQUEST, exampleRequest)]);
```

O **takeLatest** fica escutando a action **BOTAO_CLICADO_REQUEST**. Quando ela chega, ele chama a **exampleRequest**.

Mas tem um detalhe inteligente: se o usuário clicar no botão várias vezes rapidamente, o **takeLatest** cancela as anteriores e processa só a última. É como um elevador que, se você apertar vários andares em sequência, só executa o último comando. Evita requisições duplicadas desnecessárias.

O **all** simplesmente agrupa todas as sagas do módulo num array — quando o projeto crescer e tiver várias sagas, todas ficam listadas aqui.

O reducer.js — O Cartório

A Saga fez o trabalho assíncrono e disparou uma action de resultado. Agora essa action chega no **reducer** — o cartório que decide como o estado muda.

```
js
const initialState = {
  botaoClicado: false,
};

export default function(state = initialState, action) {
  switch (action.type) {
    case types.BOTAO_CLICADO_REQUEST: {
      console.log('Estou fazendo a requisição');
      return state;
    }
    case types.BOTAO_CLICADO_SUCCESS: {
      console.log('Sucesso!');
      const newState = { ...state };
      newState.botaoClicado = !newState.botaoClicado;
      return newState;
    }
    case types.BOTAO_CLICADO_FAILURE: {
      console.log('Um erro ocorreu na operação');
      return state;
    }
    default: {
      return state;
    }
  }
}
```


O initialState — A ficha em branco

Todo reducer começa com um estado inicial — a ficha zerada do paciente antes de qualquer coisa acontecer. Aqui é simples: **botaoClicado: false**.

O switch — O atendente do cartório

O reducer recebe todas as actions que passam pelo sistema. O **switch** é o atendente que olha o tipo da action e decide o que fazer:

- **REQUEST** chegou? Não muda nada no estado, só loga no console. A Saga já está cuidando disso.
- **SUCCESS** chegou? Emite um documento novo — copia o estado atual com `...state` e inverte o valor de `botaoClicado`.
- **FAILURE** chegou? Devolve o estado sem alteração.

Qualquer outra coisa? O default devolve o estado intacto.

O ...state — A cópia do documento

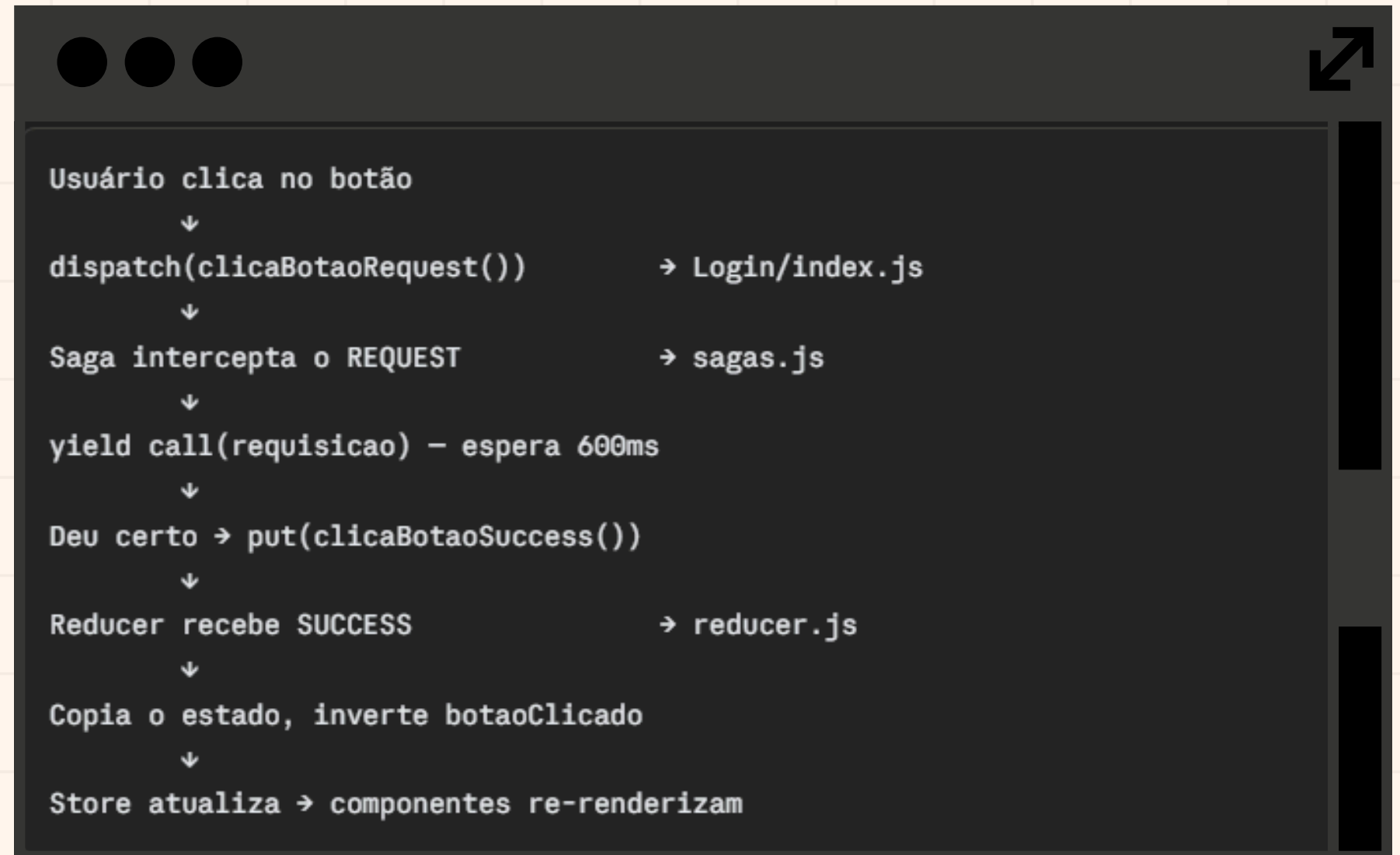
Repara nesse detalhe importante:

```
js
const newState = { ...state };
newState.botaoClicado = !newState.botaoClicado;
return newState;
```

O reducer nunca modifica o estado **original** — ele tira uma cópia com o spread operator **...state** e modifica a cópia. É a regra de ouro do Redux: imutabilidade. Assim como um cartório não rasura o documento original, o reducer sempre emite um documento novo.

O fluxo completo — Do clique ao estado

Agora você tem todas as peças. Vamos ver o ciclo completo de uma vez:



Sete passos para um clique de botão. Parece muito, mas cada passo tem uma responsabilidade clara e isolada — e quando o projeto escalar pra dezenas de funcionalidades, essa organização vai salvar sua sanidade.

CAPÍTULO 6

A CENTRAL COMPLETA (CONECTANDO TUDO)

Chegamos ao último andar do subsolo. Até agora vimos as peças individuais — o reducer do **example**, a saga do **example**. Agora vamos ver os três arquivos que **conectam tudo num único sistema**.

O rootReducer.js — O painel central

```
js

import { combineReducers } from 'redux';
import exampleReducer from './example/reducer';

export default combineReducers({
  example: exampleReducer,
});
```

Lembra que o Redux tem um único **store** — uma única ficha cadastral da clínica? Mas uma clínica tem fichas de pacientes, de médicos, de consultas... Como organizar tudo num único objeto sem virar uma bagunça?

O **combineReducers** resolve isso. Ele pega todos os reducers do projeto e os organiza como gavetas de um arquivo:

```
js

// O estado global fica assim:
{
  example: {
    botaoClicado: false
  }
  // futuramente:
  // auth: { isLoggedIn: false, token: null }
  // pacientes: { lista: [], loading: false }
}
```

Cada módulo cuida da própria gaveta. O reducer **example** só lê e escreve em **example**. O reducer **auth** só mexe em **auth**. Nunca um interfere no outro.

Quando o projeto crescer, é só adicionar uma linha:

```
js

export default combineReducers({
  example: exampleReducer,
  auth: authReducer,      // novo módulo
  pacientes: pacientesReducer, // novo módulo
});
```

O rootSaga.js — A central de escuta

```
js
export default function* rootSaga() {
  return yield all([example]);
}
```

O mesmo conceito, mas para as Sagas. Cada módulo tem suas próprias sagas escutando suas próprias actions. O **rootSaga** é o **supervisor** que inicializa todas elas de uma vez.

O **all** faz todas rodarem em **paralelo** — a saga do **example** escuta o que é do **example**, a saga do **auth** escuta o que é do **auth**, sem interferência.

Quando crescer:

```
js
export default function* rootSaga() {
  return yield all([example, auth, pacientes]);
}
```

O reduxPersist.js — O arquivo permanente

```
js
export default (reducers) => {
  const persistedReducers = persistReducer(
    {
      key: 'REACT-BASE',
      storage,
      whitelist: ['example'],
    },
    reducers
  );
  return persistedReducers;
};
```

Esse arquivo configura o que a clínica **guarda** e o que **descarta** ao fechar.

Três configurações importantes:

key: 'REACT-BASE' — o nome da entrada no **localStorage**. Se você abrir o DevTools do navegador na aba Application → Local Storage, vai encontrar uma chave chamada **REACT-BASE** com todo o estado salvo em JSON.

storage — onde salvar. Por padrão é o **localStorage** do navegador, mas poderia ser **sessionStorage** ou até AsyncStorage no React Native.

whitelist: ['example'] — e esse é o mais importante. A lista de permissões. Só as gavetas listadas aqui são salvas. Se você tiver uma gaveta auth mas não colocá-la no whitelist, ela some ao recarregar a página.

O oposto seria o **blacklist** — lista de gavetas que você quer excluir da persistência. Por exemplo, um estado de **loading** não precisa ser salvo.

O MAPA COMPLETO DA CENTRAL

Agora você tem a visão completa de como tudo se conecta:

```
store/
├─ index.js      → monta a central (store, middlewares, persistor)
├─ modules/
│   ├─ types.js   → nomes das mensagens
│   ├─ rootReducer.js → combina todas as gavetas
│   ├─ rootSaga.js → inicializa todos os escutadores
│   ├─ reduxPersist.js → define o que é salvo no localStorage
│   └─ example/
│       ├─ actions.js → as mensagens do interfone
│       ├─ reducer.js → o cartório que altera o estado
│       └─ sagas.js → o corredor que chama a API
```

Quando você criar o módulo auth para o login real com sua API REST, vai criar uma pasta auth/ com exatamente esses mesmos quatro arquivos — e registrar no rootReducer e no rootSaga. O projeto já está preparado pra crescer.

O ciclo completo, agora com tudo no mapa

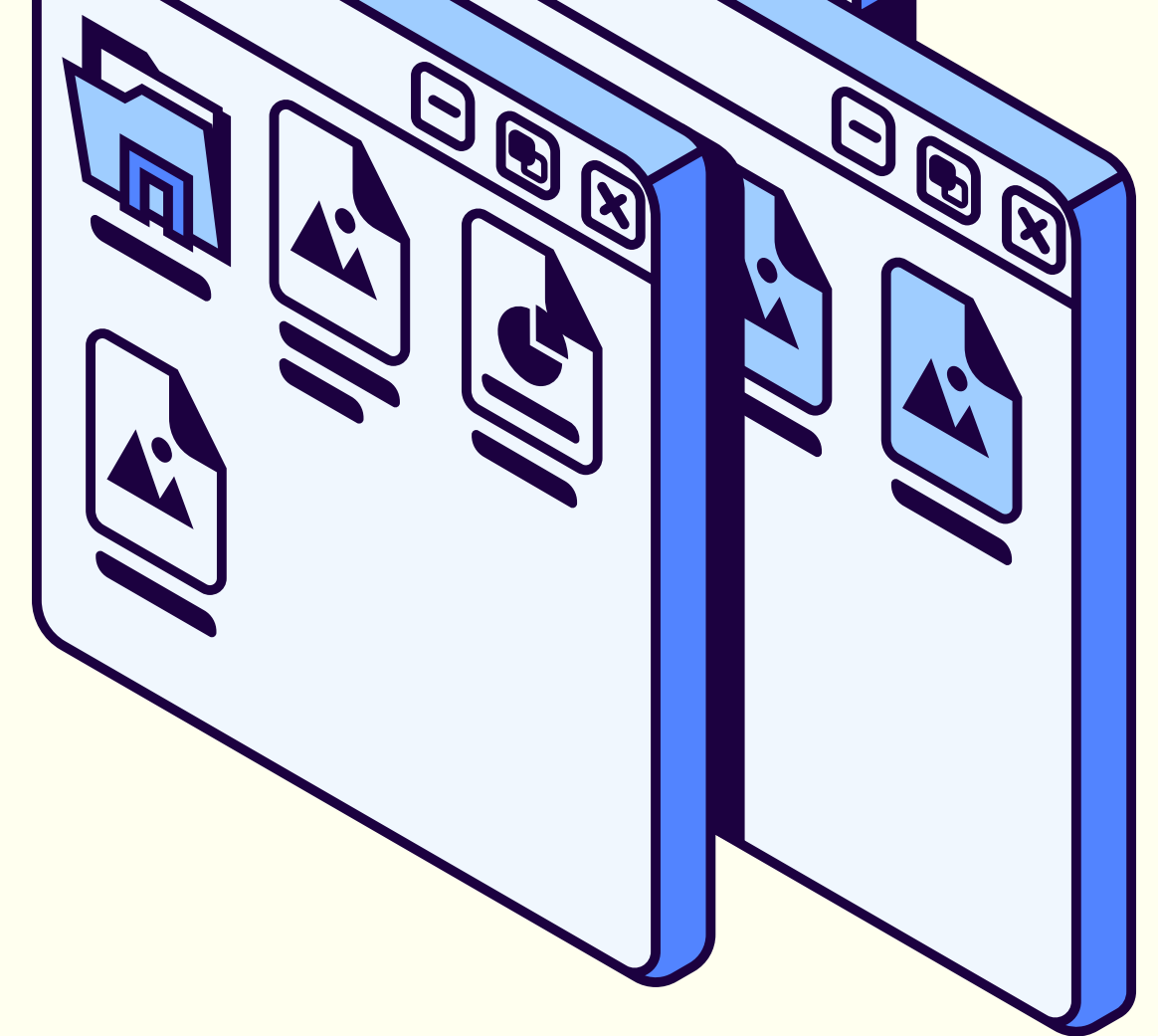
```
Login clica botão
    ↓
dispatch(clicaBotaoRequest())
    ↓
sagaMiddleware intercepta
    ↓
rootSaga → example saga escuta BOTAO_CLICADO_REQUEST
    ↓
yield call(requisicao) — chama a API
    ↓
yield put(clicaBotaoSuccess())
    ↓
rootReducer → example reducer recebe SUCCESS
    ↓
Copia estado, atualiza botaoClicado
    ↓
reduxPersist salva no localStorage
    ↓
Store atualiza → tela re-renderiza
```

POR QUE SERVE COMO BASE?

Porque o projeto já resolve os problemas chatos antes mesmo de você começar a construir qualquer funcionalidade.

Pensa assim: toda aplicação React que se conecta a uma API vai precisar das mesmas coisas — um lugar para guardar o estado global, uma forma de fazer requisições autenticadas, um sistema de rotas com páginas protegidas, notificações de sucesso e erro, estilos organizados e dados que persistem ao recarregar a página. São problemas universais.

Essa base já entrega tudo isso pronto, testado e integrado. Quando você for construir o sistema de alunos da sua API REST, por exemplo, não vai precisar pensar em infraestrutura



NOTAS

O conceito de Estado:

No contexto do React e do Redux, estado é qualquer informação que a aplicação precisa lembrar para funcionar corretamente.

Pensa assim: uma página web não é estática — ela reage a coisas. O usuário está logado ou não? O botão foi clicado? A lista de pacientes já carregou? Tem um erro na tela? Essas respostas mudam ao longo do tempo, e alguém precisa guardá-las. Esse "alguém" é o estado.

Minha definição: “o estado é o valor atual ocupado num campo de uma variável.”

A distinção importante é entre dois tipos:

Estado local — pertence a um componente só. Um campo de texto que você está digitando, por exemplo. Só aquele componente precisa saber. O React gerencia isso com `useState`.

Estado global — precisa ser acessado por vários componentes ao mesmo tempo. Se o usuário está logado, o Header precisa saber para mostrar o nome dele, a rota precisa saber para liberar o acesso, o axios precisa saber para anexar o token. Esse é o trabalho do Redux.

Resumindo numa frase: estado é a memória viva da aplicação — tudo que ela precisa lembrar para decidir o que mostrar e como se comportar em cada momento.

O estado é o valor atual de uma variável, mas uma variável especial: quando ela muda, a interface reage automaticamente. Não é você quem vai lá atualizar o HTML na mão — o React percebe a mudança e re-renderiza o que for necessário.

O modelo de projeto ainda é atual?

Embora o ano de publicação desse segmento do curso seja 2020, o projeto pode ser considerado funcional, mas tem partes que estão sendo substituídas no mercado. O que ainda é muito atual?

- React — segue dominante no mercado
- Styled Components — ainda muito usado, mas concorrido
- Axios — padrão absoluto para chamadas HTTP
- React Router — continua sendo a solução de rotas

O que está sendo substituído

Redux + Redux Saga é o ponto mais importante aqui. Essa combinação foi o padrão por muitos anos, mas é verbosa — você viu isso na prática: criar uma action simples exige `types.js`, `actions.js`, `reducer.js` e `sagas.js`.

Hoje o mercado está migrando para: Redux Toolkit – é o próprio Redux simplificado. Essa ferramenta reduziria 70% do código escrito no projeto. React Query / TanStack Query – Substitui o Saga para chamadas de API simplificadas. Zustand, uma alternativa ao Redux, muito mais leve.

NOTAS

Mas... Ainda vale a pena estudar essa base?

Sim, e muito. Por algumas razões:

Primeiro, muitas empresas ainda têm projetos grandes rodando exatamente com essa stack — você vai encontrar isso no mercado.

Segundo, e mais importante: quem entende Redux **"na raiz"** entende qualquer gerenciador de estado. O Redux Toolkit, o Zustand, o React Query — todos ficam triviais pra quem dominou o fluxo action → saga → reducer.

É como aprender a dirigir num carro manual antes do automático.